论文解读 · 具身智能 / Embodied RSI

PyRUA-REAL

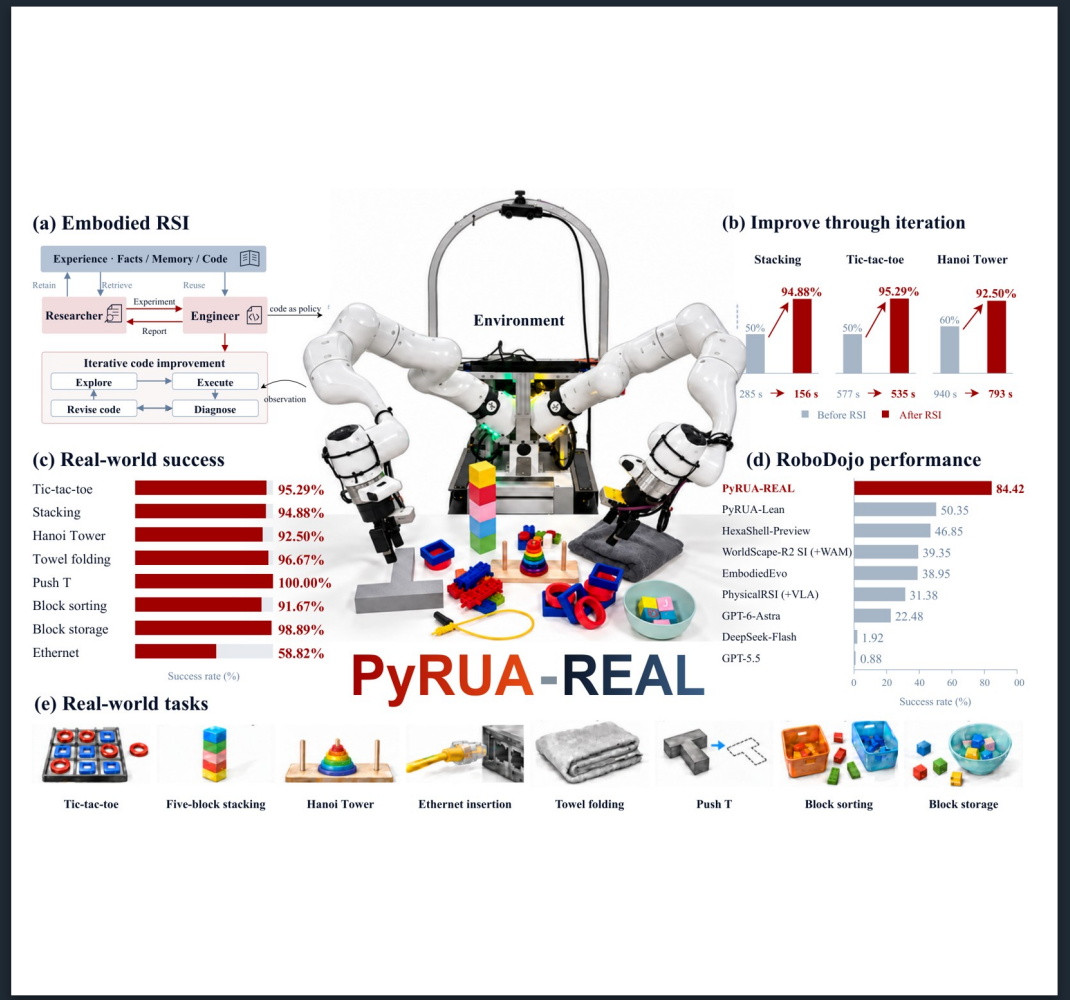
Robot Exploration and Adaptive Learning through RSI in the Real World

让编程智能体在真机上试错、改代码，把经验固化成免大模型运行的机器人程序

Qiang Wang*, Jianxin Bi* ‡, Shulong Jiang*, Ruiyang Si, Wenbo Huang, Kaiqi Chen, Jiaxin Li, Liyun Li, Xuanyi Li †, Daquan Zhou †

北京大学 · EngineAI · 新加坡国立大学

arXiv 2611.06768 | 首次提交 2026-11-04



图源：论文 Figure 1

ONE-LINER

一句话总结：真机试错 → 经验沉淀 → 冻结成免大模型的程序

PyRUA-REAL 让「研究员 + 工程师」两个编程智能体直接在真实双臂 Franka 上做实验、诊断、改代码，把交互经验存成「事实 / 经验教训 / 可复用代码」三层记忆，最终把任务固化为**运行时不调用大模型**的程序。

7 / 8
真机任务得到
免智能体运行的程序

91.1%
8 个真机任务
开发期平均成功率 *

84.4%
RoboDojo 42 任务成功率
（PyRUA-Lean 50.35%）

5–11×
执行速度快于
在线编码 / 在线动作推理

* 91.1% 是 7 个离线任务加上网线插入「在线辅助」成功率 58.82% 后的平均；且成功率是把开发过程中多个代码版本的尝试合并统计，不等于同一冻结程序的重复测试可靠性。

RESEARCH QUESTION

核心问题：如何把物理交互经验变成持续变强的机器人能力?

背景

GPT-6-Astra 这类编程智能体能把空间推理写成可执行代码，并根据执行反馈修改。

论文追问：物理交互经验能否在模型参数固定的前提下，变成跨次尝试持续保留、不断改进的机器人能力?

形式化

$$a_t = \pi_\tau(o_{\leqslant t}; \mathcal{L})$$

任务程序 π_τ + 共享技能库 $\mathcal{L}$

$$(\pi^{k+1}, \mathcal{L}^{k+1}, \mathcal{M}^{k+1}) = U_\theta(\tau, \pi^k, \mathcal{L}^k, \mathcal{M}^k, E^k)$$

θ 固定； E^k 为执行证据； $\mathcal{M}$ 为经验库；在交互和算力预算内最大化期望表现

① 决策难

该做哪个实验？改哪处实现？何时放弃当前路线？反复局部修改可能保留了本就不合适的感知或控制方案。

② 反馈有歧义

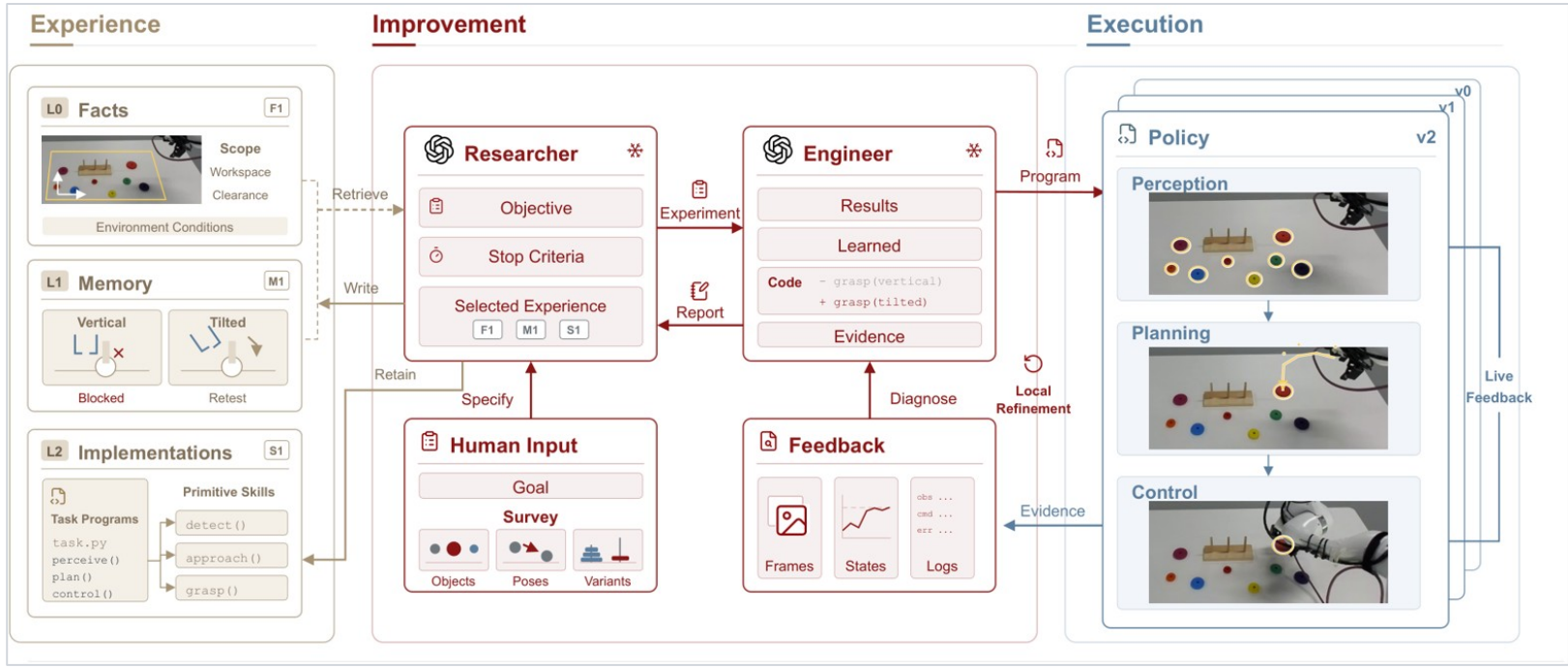
一次「失败」可能来自运动、感知，或者来自判断成败的验证程序本身。

③ 泛化与复用

一种配置下成功 ≠ 对位置、朝向、任务变体鲁棒；成功实现能否迁移到新任务、能否脱离在线智能体运行?

METHOD • SYSTEM

框架：研究员定路线、工程师做实验，经验沉淀、程序独立执行



Improvement 改进

研究员：目标 / 停止条件 / 精选经验

工程师：结果 / 发现 / 代码 / 证据

Experience 经验

L0 事实 • L1 记忆 • L2 实现

成功和失败的发现都会保留

Execution 执行

冻结程序：感知 → 规划 → 控制

运行时不调用大模型，靠实时反馈闭环

人类只给：任务目标 goal ＋ 变化范围说明 survey；标定、控制器和求解器均需智能体自行建立。

METHOD • COLLABORATION & MEMORY

以「实验」为协作单位，经验分三层沉淀

研究员 Researcher

看整个任务与所需变体

选技术路线；决定打磨、换方向或进入下一阶段

→

实验规格 Experiment

目标

停止条件：达成 / 假设被否 / 遇瓶颈 / 连续无进展

精选经验：事实、历史方案、技能

→

工程师 Engineer

写代码、上真机、局部测试与诊断

可只测某一步，不必每次跑全任务

→

报告 Report

results：结果与终止原因

learned：发现

evidence：执行记录

code：最终实现

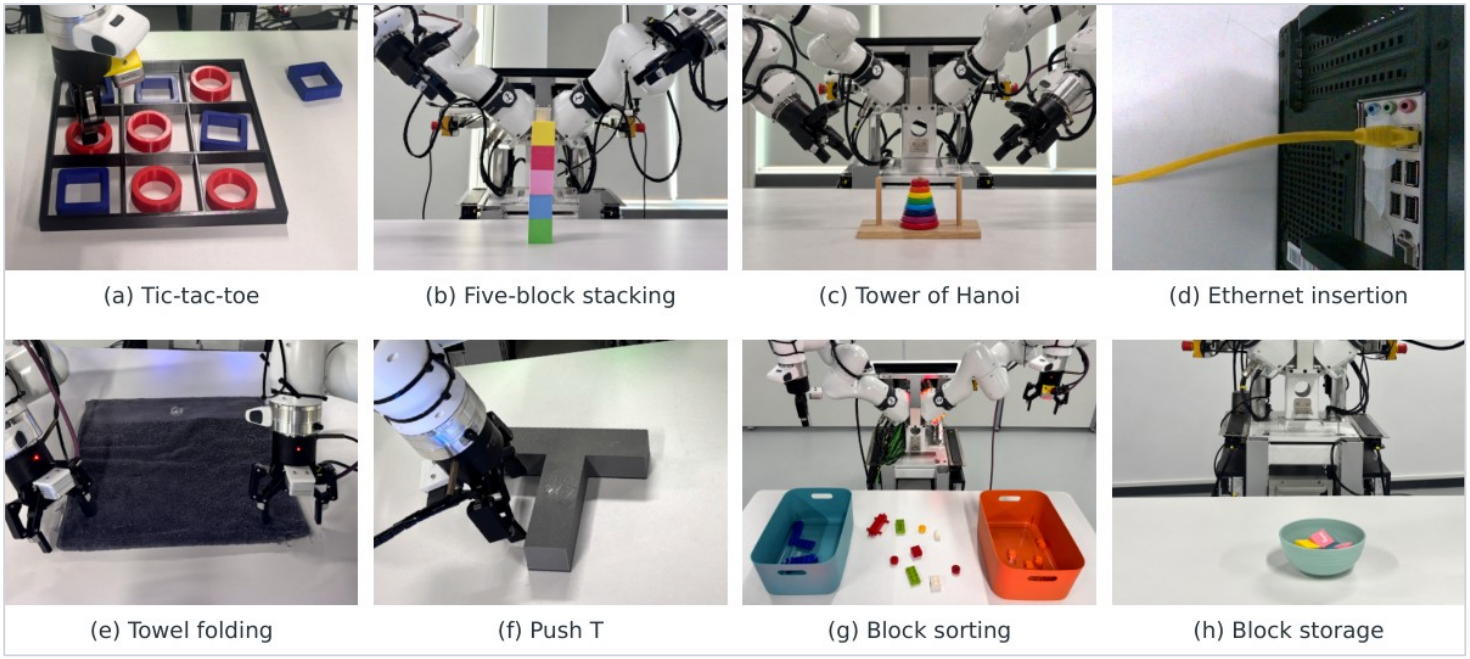
↺ 研究员读报告 → 继续打磨当前方案 / 改变方向 / 进入下一阶段，并据此定义下一个实验

L0 事实 Facts	L1 记忆 Memory	L2 实现 Implementation
存储： 机器人、工作空间、运行条件 **用途：** 研究员：理解相关环境条件	**存储：** 遇到的问题、尝试过的方案、结果、失败原因 **用途：** 研究员：检索相关的历史任务、方案与技能	**存储：** 任务程序与可复用原语（detect / approach / grasp…） **用途：** 工程师：查看、改编、测试实现

技能使用次数只按每次实验的最终实现计算；由研究员决定哪些改编后的实现保留为新技能，并记录适用范围与局限。

SETUP

8 个真机任务覆盖推理、长时程、精密接触、柔性物体与双臂协作



(a) 井字棋 (b) 五块堆叠 (c) 汉诺塔 (d) 网线插入 (e) 叠毛巾 (f) Push T (g) 分拣 (h) 收纳

真机平台

2× Franka Research 3 ＋ Robotiq-85 夹爪

头部 ZED Mini ＋ 腕部 RealSense D405

精密	堆叠 · 汉诺塔 · 网线 · Push T
长时程	井字棋 · 堆叠 · 汉诺塔
推理	井字棋 · 分拣
接触丰富	网线 · Push T
柔性 / 双臂	叠毛巾
多物体	井字棋 · 堆叠 · 汉诺塔 · 分拣 · 收纳

仿真： RoboDojo 42 任务

泛化 12 · 记忆 6 · 精细 8 · 长时程 8 · 开放 8

7/8 个任务得到免智能体程序；首次成功耗时 1.5 h 到 45 h 不等

任务	首次成功耗时	开发期成功率	执行时长中位数
井字棋	~9 h	**95.29%**	539 s
五块堆叠	~2 h	**94.88%**	179 s
汉诺塔	~22 h	**92.50%**	1003 s
叠毛巾	~5 h	**96.67%**	30 s
Push T	~45 h	**100%**	129 s
分拣	~4 h	**91.67%**	179 s
收纳	~1.5 h	**98.89%**	109 s
网线插入（纯离线）	240 h 未成功	**0%**	—
网线插入（在线 Astra 辅助）	再加 ~15 h	**58.82%**	519 s

91.1% 是怎么算的?

= (7 个离线任务成功率
＋ 网线在线辅助 58.82%) ÷ 8

≈ 83.7%

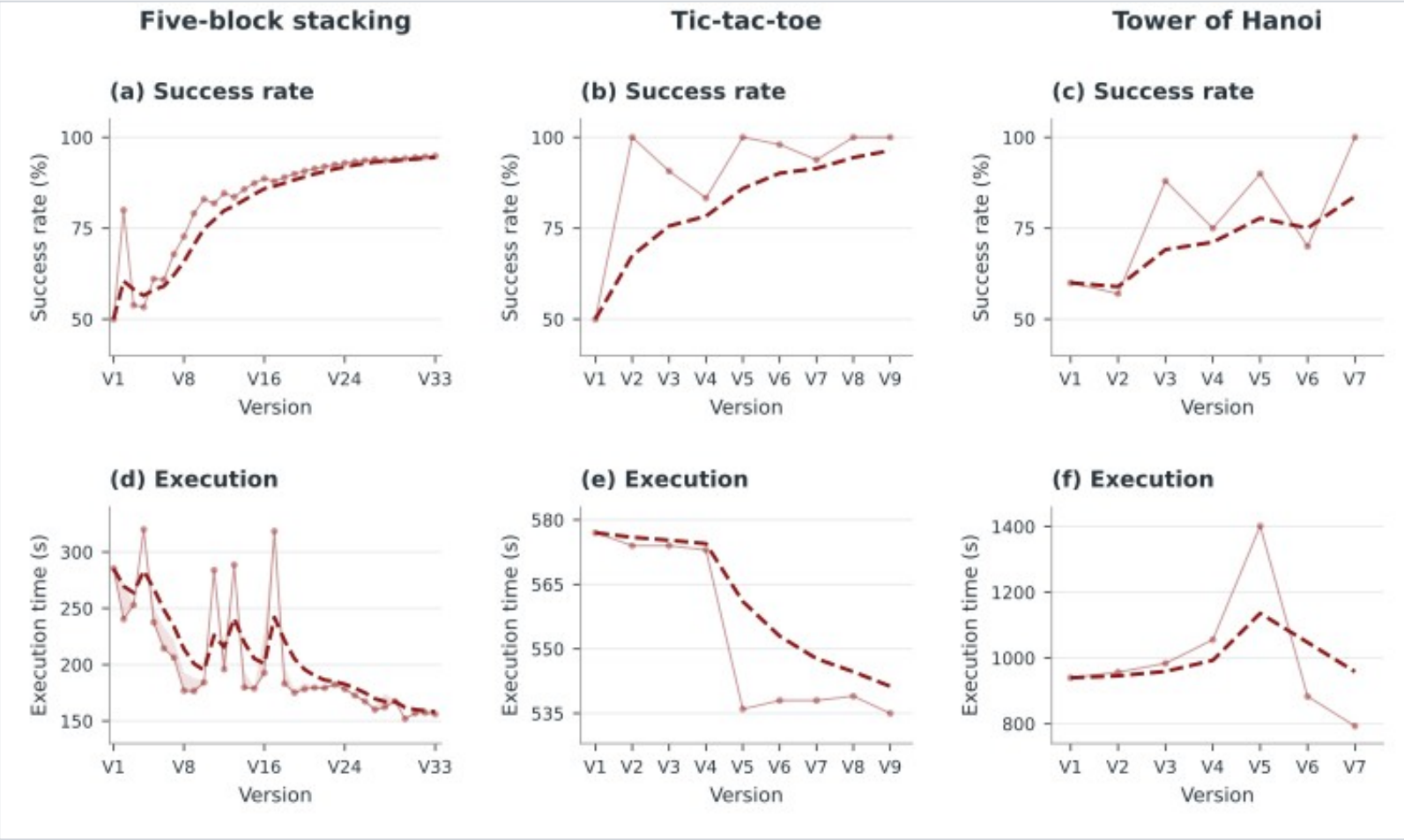
网线插入按纯离线 0% 计

≈ 95.7%

只看 7 个离线任务

所有成功率都是把开发过程中多个代码版本的尝试合并统计，不等于同一冻结程序的可靠性。

首次成功后仍在进步：堆叠循环时间 285 s → 156 s



五块堆叠 • 渐进局部打磨

循环时间 285.3 → 156.0 s

横向误差 2.69 → 0.44 mm；角度 1.37° → 0.25°

阶段成功率 88.7% → 98.7%

井字棋 • 运动策略切换

V4→V5：多段高空途经点 → 单个 JointMotion

573 → 536 s；成功率 50% → 100%（中途有回落）

汉诺塔 • 先稳后快

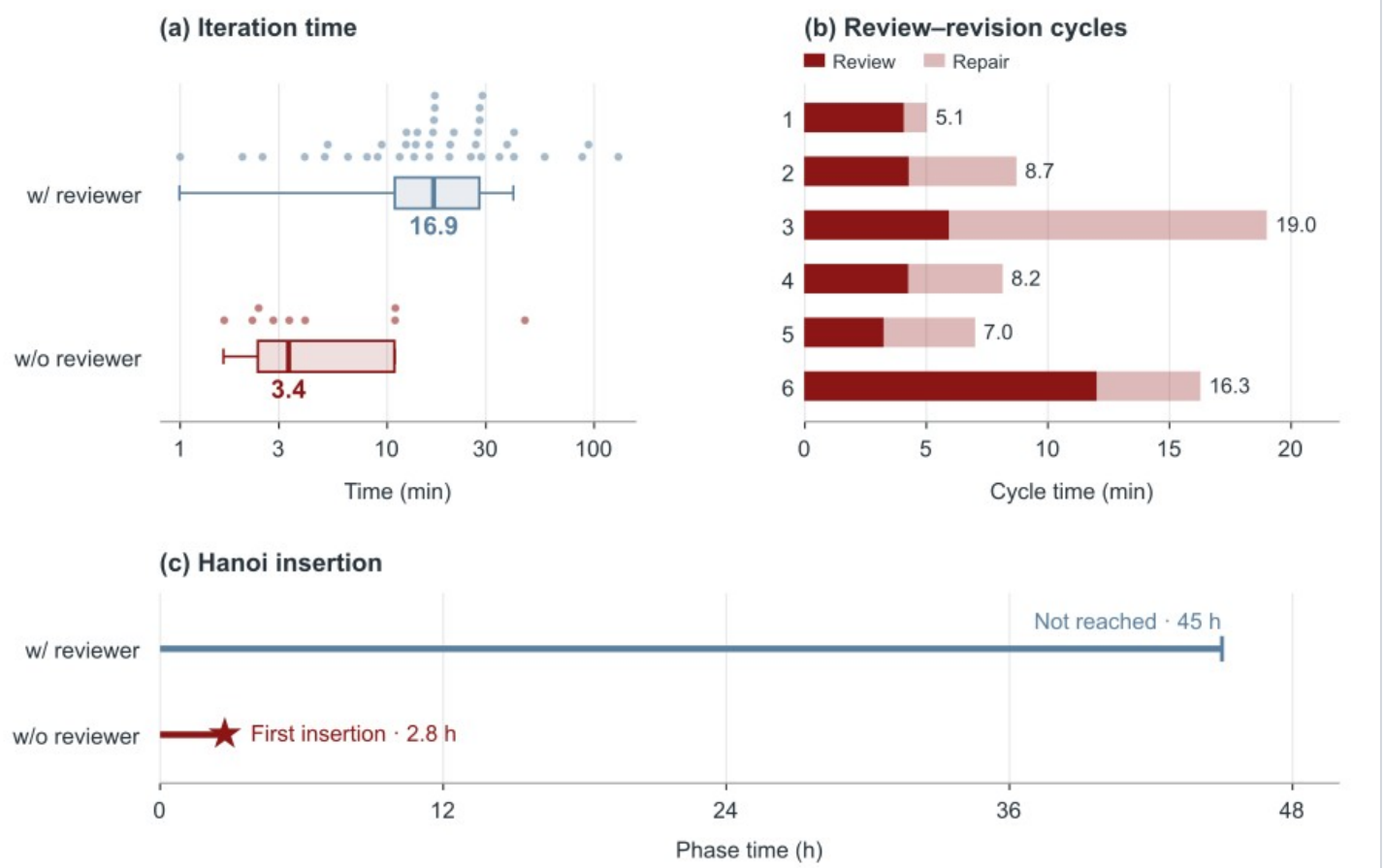
稳健阶段 1402 s → 提速版本 883 / 793 s

成功率 60% → 100%

开发过程数据，非冻结程序重复测试；误差为视觉估计。

H2 • COORDINATION COST

加一个只会否决的审稿智能体，反而拖慢探索



16.9
有审稿人
间隔中位数（min）

3.4
无审稿人
间隔中位数（min）

汉诺塔中的否决—修改周期

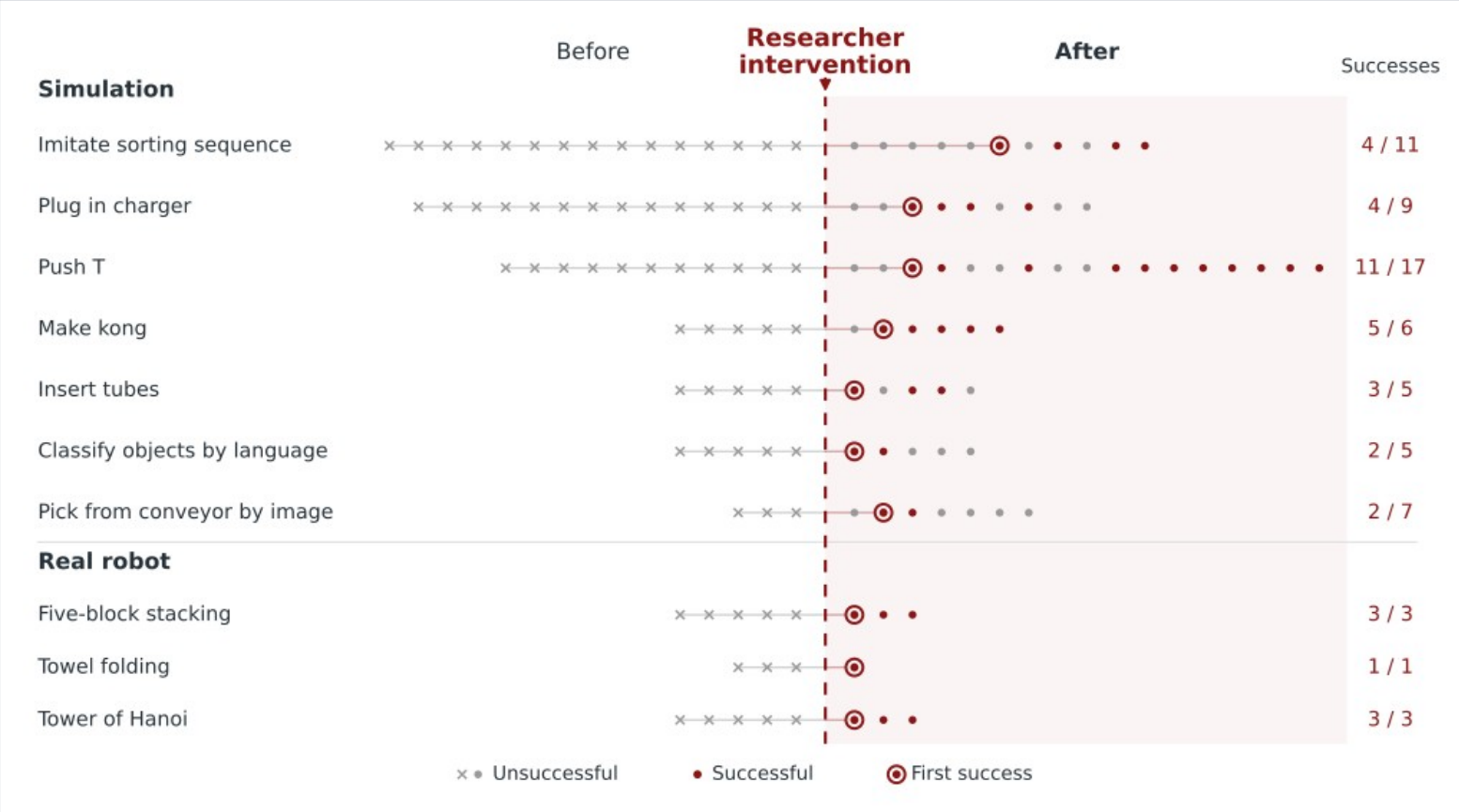
6 次，每次 5.1–19.0 min ，中位数 8.5 min

45 h 未插入 vs 2.8 h 首次插入

两阶段先后进行，后者受益于先前探索，不能单独归因于去掉审稿人。

可靠结论：审批角色降低了获取新物理证据的速度。

研究员的路线重估能打破局部修改的死循环



3/3 • 1/1 • 3/3

真机：堆叠 • 叠毛巾 • 汉诺塔
重估后的下一次试验即成功

仿真效果参差

Push T 11/17 • Make kong 5/6

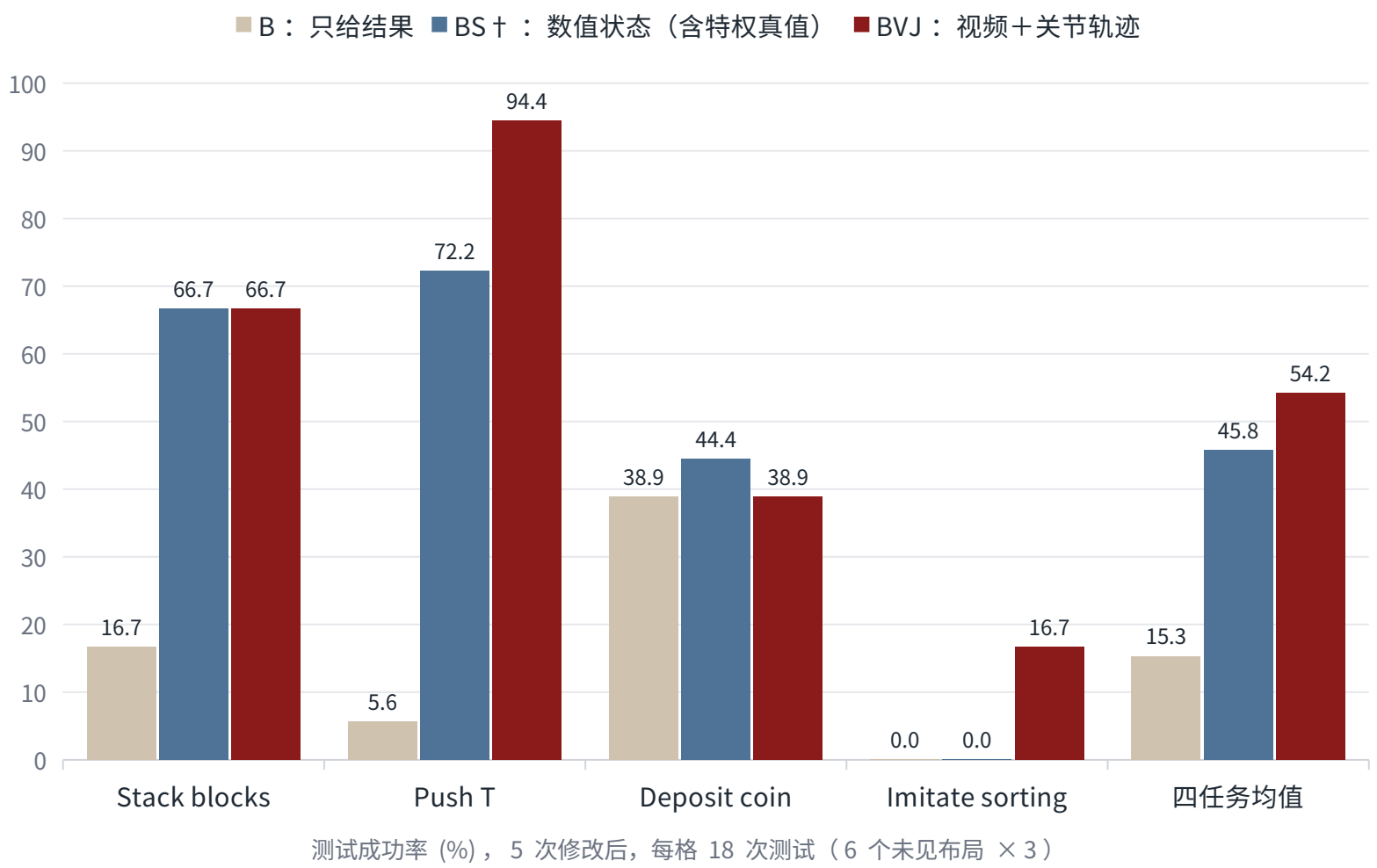
Imitate sorting 4/11 • Pick from conveyor 2/7

启示

给局部修改设上限，未解决的失败交回研究员重估路线。

（相关而非因果：未与继续探索的效果分离）

反馈越丰富，改代码越有效：视频＋关节 54.2% vs 只给结果 15.3%



测试成功率（%），5 次修改后，每格 18 次测试（6 个未见布局 × 3）

54.2% vs 15.3%

BVJ vs B 平均测试成功率

更丰富，但不更贵

优化器时间 25.4 vs 31.8 min

Token 3.65M vs 3.54M （相近）

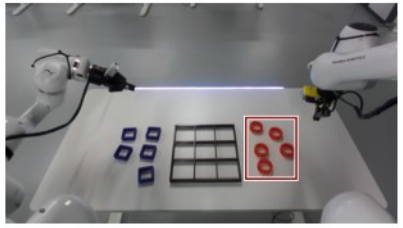
注意

收益随任务而变： Push T 5.6→94.4%，Deposit coin 持平

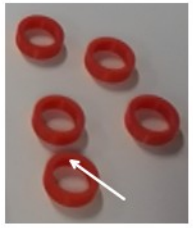
每个条件仅 1 次会话，方差未测

很多「失败」出在测量上：先分清是执行错还是验证错

Inspected image



Touching rings



Same-image replay

Before		After
4 rings	→	5 rings
80.6 mm		5.7 mm

Acceptance threshold: 25 mm

Segmentation patch · source excerpt

```
-   def colored_objects(image):
+   def colored_objects(image, split_touching=False):
        ...
+           if color == "red" and split_touching:
+               mask = split_touching_rings(mask)
            contours, hierarchy = cv2.findContours(...)
```

Height reference

Physical verification

Previous run		Next run
6 empty re-grasps	→	Grasp on first descent

Height-reference change · pseudocode (m)

```
-   plane_left = stored_plane
+   plane_left = (wrist.plane_base
+                 if wrist.rms < 0.003 and wrist.n >= 3000
+                 else stored_plane)
    target, floor = plane_left + 0.020, plane_left + 0.010
```

井字棋：检测器把两个相接的环当成一个

检出 4 个环（实际 5 个）→ 误差 80.6 mm > 阈值 25 mm，误判失败

加入相接红环的分割 → 同图重放 5 个环、5.7 mm

随后跑 50 项回归检查 + 178 帧静态感知检查

修的是验证器，机器人动作未变

叠毛巾：桌面高度参考偏高导致空抓

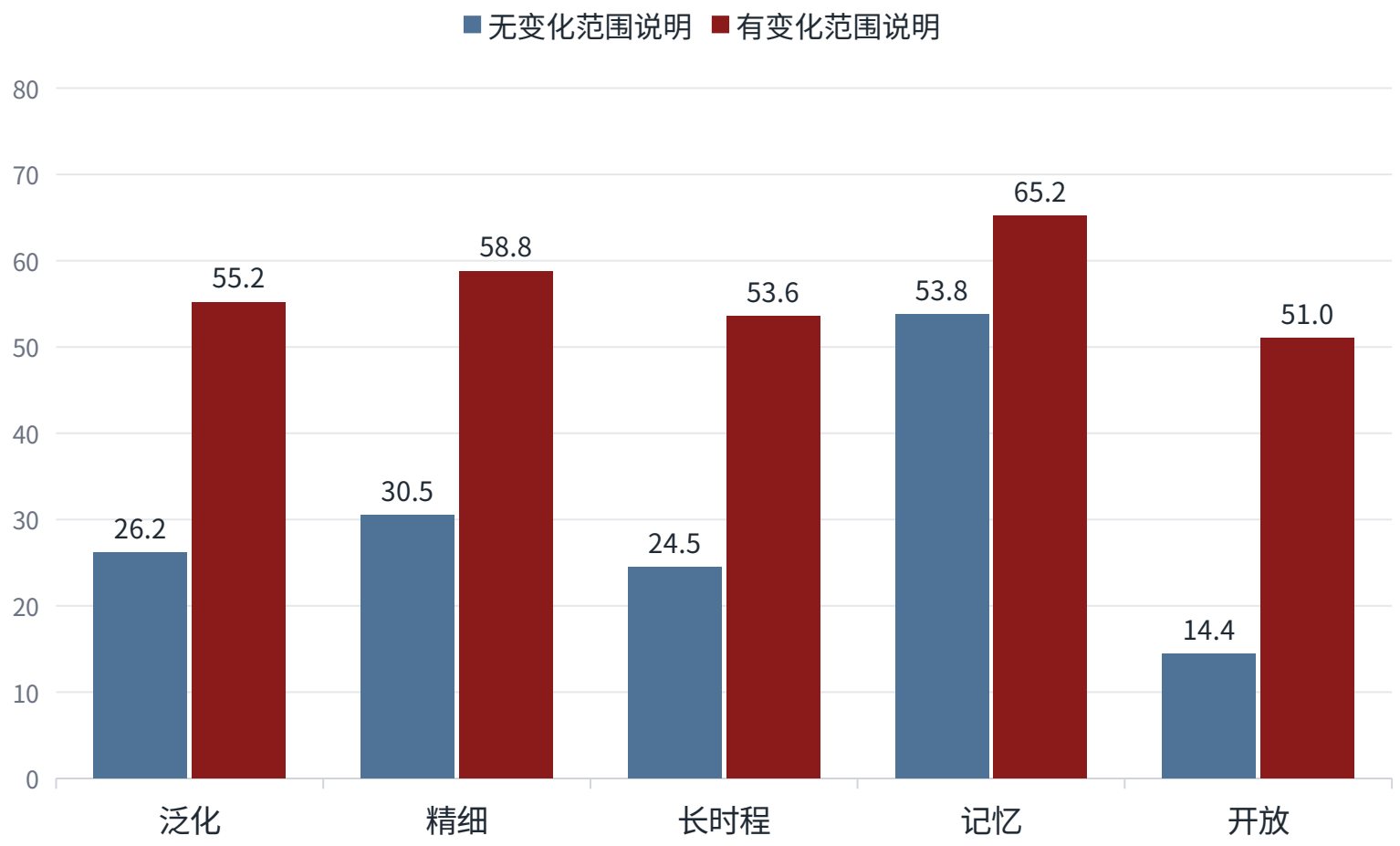
存储的平面比腕部局部拟合高 25–35 mm → 6 次重抓全空

局部拟合 RMS < 3 mm 且点数 ≥ 3000 时改用腕部平面

下一次运行首次下探即抓住（单次恢复，非重复成功率）

H4 • EXPLICIT TASK REQUIREMENTS

写明任务变化范围后，五类能力的未见布局成功率全部提升



未见布局成功率 (%)；提升（pp）：泛化 +28.9 • 精细 +28.3 • 长时程 +29.1 • 记忆 +11.4 • 开放 +36.6

+11.4 ~ +36.6 pp

五类能力全部提升

单任务例子

Stack bowls 0.0 → 86.4%

Fasten screws 9.1 → 81.8%

General pickup 0.0 → 74.4%

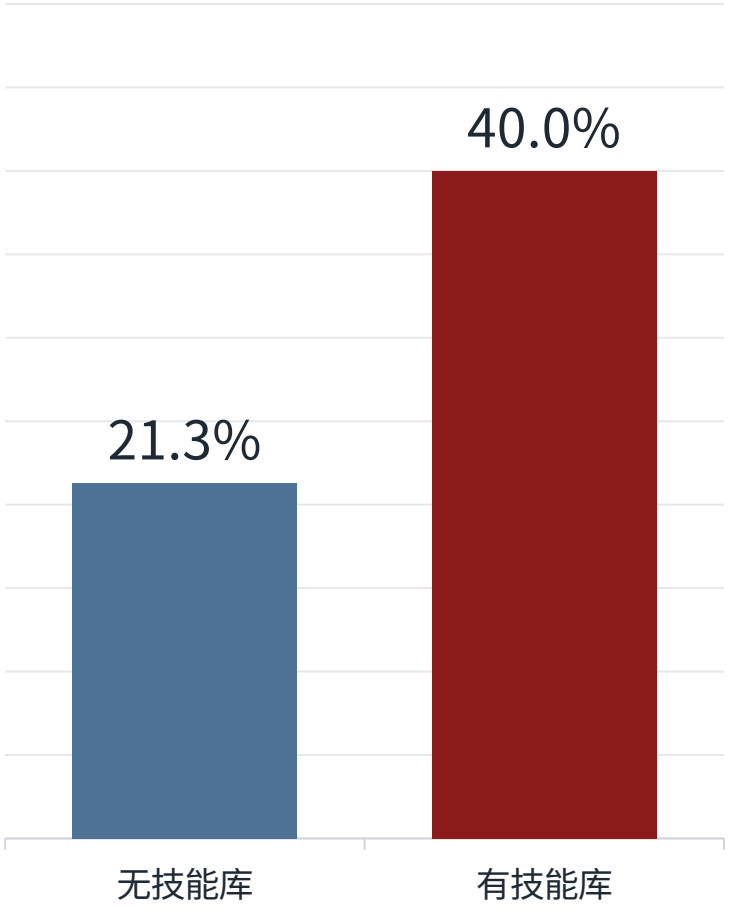
反例： Swap blocks 68.2 → 59.1%

注意

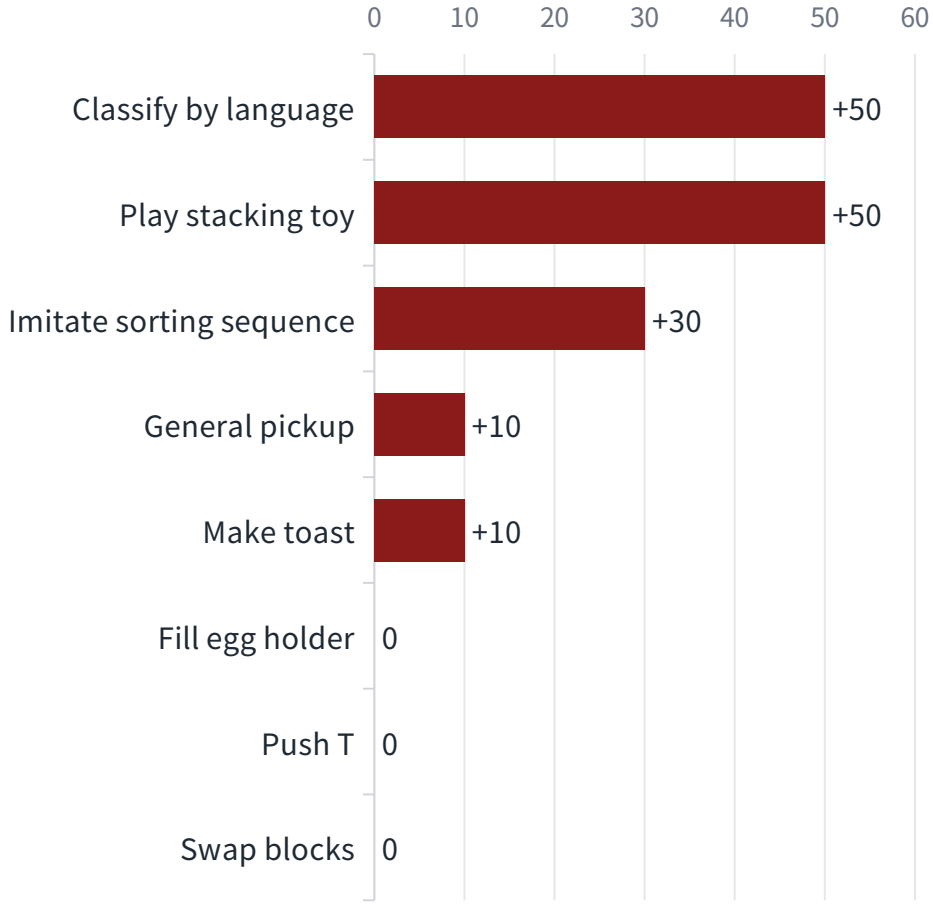
后一版本同时包含额外探索，未单独剥离规格说明的作用（32 个任务）

H5 • CROSS-TASK TRANSFER

复用旧任务的原语技能库：新任务平均成功率 21.3% → 40.0%



8 个目标任务的平均成功率



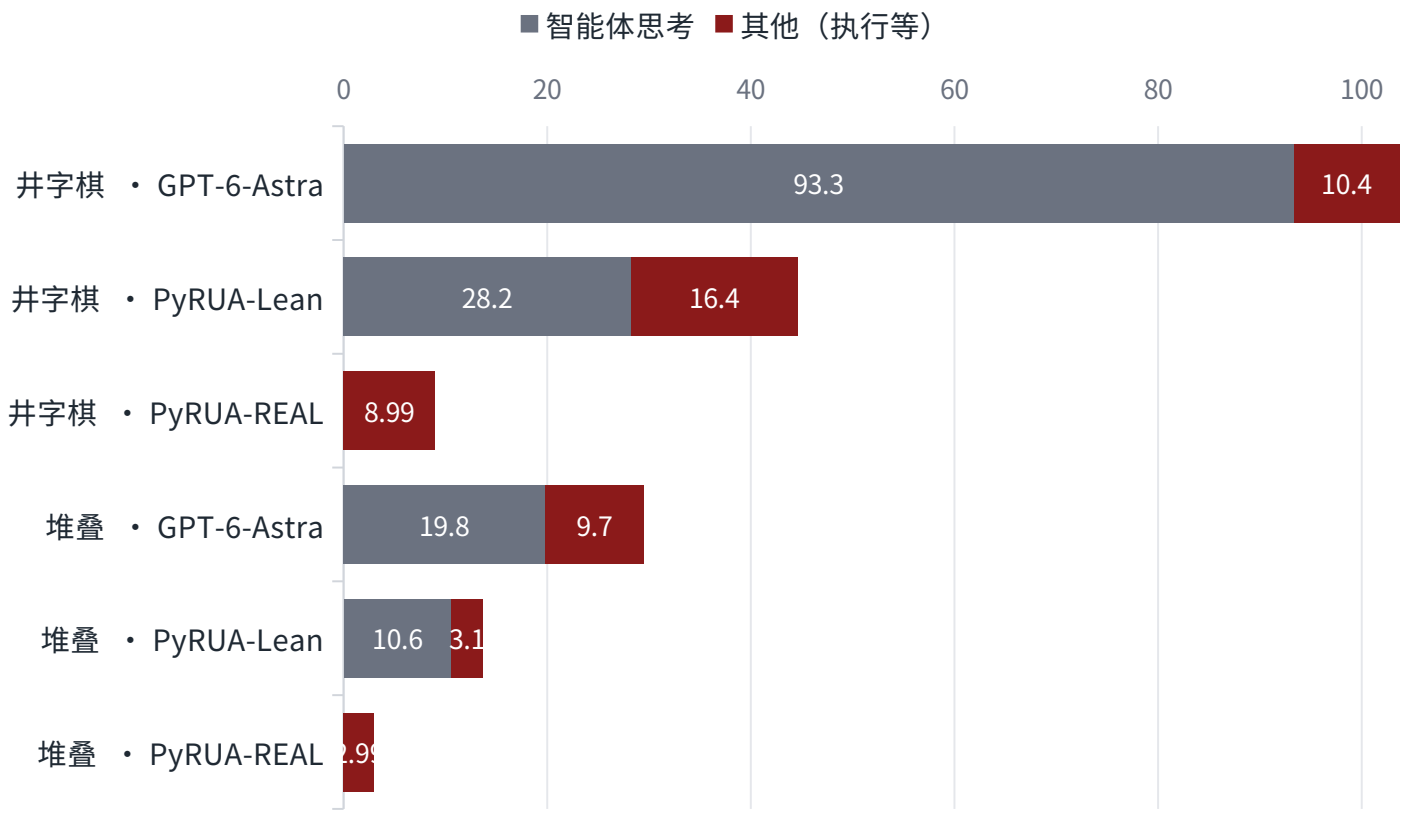
各任务成功率增益 = 有库 − 无库（百分点）

5 / 8
个新任务受益

0
个任务出现
负迁移

收益因任务而异：语言分类、堆叠玩具 +50 pp； Push T 等 3 个任务无变化。

固化成程序后执行快 5–11 倍，在线时间多耗在等待思考



执行耗时（分钟，含复位）；离线为成功运行中位数，在线为记录时间；不含开发时间

汉诺塔：两个在线基线 120 min 内均未完成； PyRUA-REAL 离线程序 16.72 min

4.6–5.0×
PyRUA-Lean
耗时倍数

9.9–11.5×
GPT-6-Astra
耗时倍数

63–90%
在线执行时间中「智能体思考」的占比

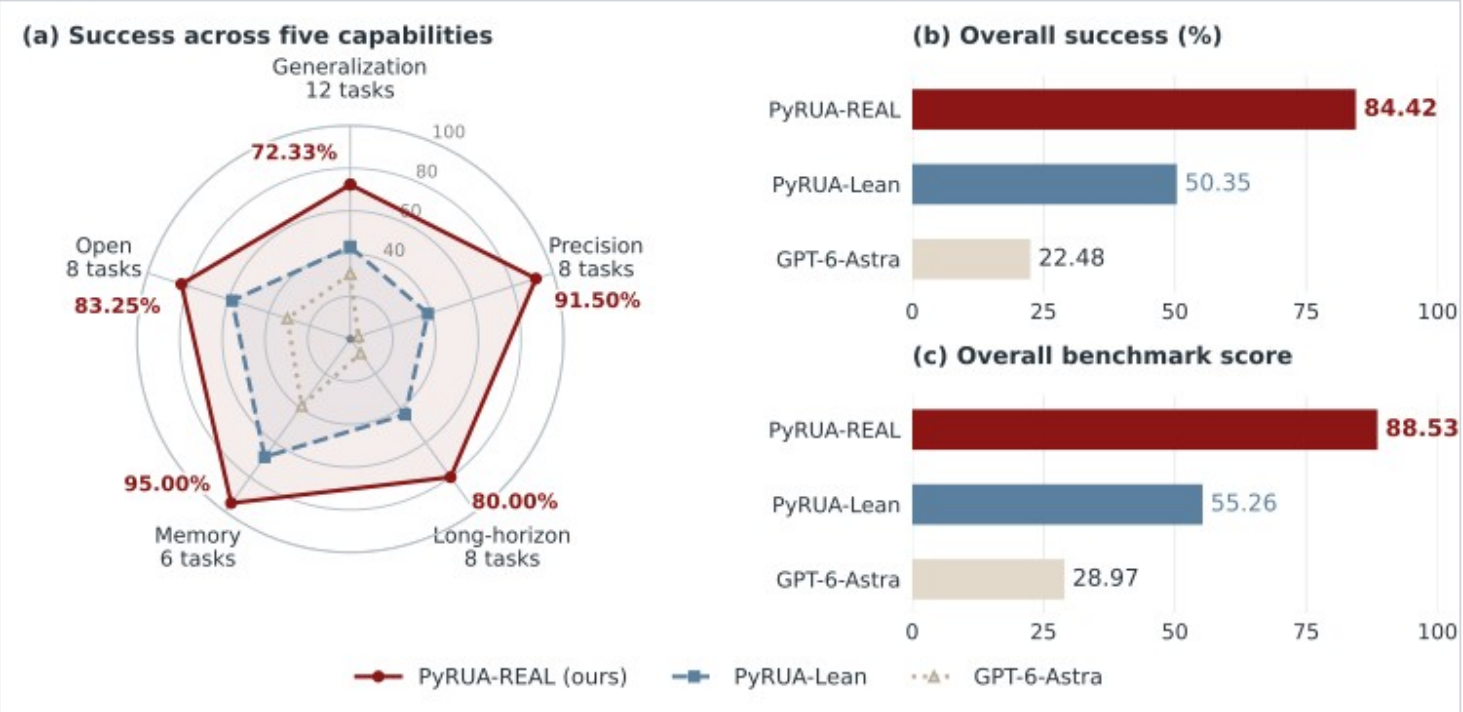
反例：网线插入仍需在线兜底

纯脚本 0/4 vs 在线 GPT-6-Astra Medium 辅助 10/17

一次成功运行：动作 281.8 s，阶段间检查与决策 220.8 s （占 44%，含人工复核）

BENCHMARK

RoboDojo 42 任务： PyRUA-REAL 84.42% vs PyRUA-Lean 50.35%



方法	泛化	精细	长时程	记忆	开放	平均
PyRUA-REAL	**72.33**	**91.50**	**80.00**	**95.55**	**88.24**	**84.42**
PyRUA-Lean	43.17	38.25	43.75	68.33	58.25	50.35
HexaShell-Preview	34.50	32.00	31.50	81.00	55.25	46.85
GPT-6-Astra	30.50	4.00	8.25	38.67	31.00	22.48

84.42% / 88.53

平均成功率 / 基准分（五类等权）

优势最大处

精细 91.50 vs 38.25

长时程 80.00 vs 43.75

（按 Figure 8 与 Table 5 ）

需谨慎

正文 §4.3.3 写精细 74.00 、长时程 56.00 、泛化 41.17 ，与图表不符

冻结程序 vs 在线零样本基线；种子不同或未注明

DISCUSSION

三点讨论：方法视野、人类介入、在线兜底

01

方法视野会限制自我改进

深度与标定误差限制了几何定位：堆叠改用局部视觉对齐＋接触反馈，汉诺塔用粗定位＋腕部相机。

General pickup 在人类提示 SigLIP 后才提升。

→ 反复失败时，显式比较替代的感知 / 控制策略。

02

人类介入会塑造习得的技能

人类帮助解决了一次智能体反复绕开错误接触检测的情况。

无人帮助时，汉诺塔智能体发明「把难抓的盘推到可抓位置」，后被井字棋复用。

→ 报告「自主改进」须说明人类贡献了什么。

03

离线程序仍需在线兜底

冻结程序只能处理其编码过的感知、控制与恢复情形。

网线插入的成功仍依赖阶段间的在线决策。

→ 离线高效执行 + 必要时调用在线智能体；何时调用仍是开放问题。

ASSESSMENT

评价：扎实的真机 RSI 实证，但头条数字需要看清口径

亮点

+ 真机上完整的「探索 → 改进 → 部署」闭环，而非只把真机当最终验证

+ 研究员 / 工程师分层协议 + 三层经验库 + 离线冻结程序，范式清晰可复用

+ 实证量少见：数十至上百小时的真实开发记录与诊断轨迹

+ 反直觉发现有价值：审批角色拖慢进度；很多失败出在验证器；部署时间主要耗在等待思考

+ 写作诚实：多处主动说明非对照、视觉估计、跨版本汇总

局限与疑点

– 91.1% 含在线辅助结果，且为跨版本汇总；缺少真机冻结程序的系统重复测试

– RoboDojo 数字前后不一致（84.4 / 84.82 / 84.42；正文分项与图表不符；种子说明矛盾）

– 对比不对等：逐任务离线开发的冻结程序 vs 在线零样本基线

– H2 先后阶段、 H4 混入额外探索、 H3 每条件 1 次会话：均为初步证据

– 成本高： Push T 首次成功 ~45 h，网线插入 240 h 未成功，且有人类提示

TAKEAWAYS

组织具身 RSI 的五条经验

1	**给丰富的执行反馈**	视频＋关节轨迹让修改更有效，也更省时（54.2% vs 15.3%）
2	**写清任务变化范围**	给出物体、布局、位姿范围，引导程序面向鲁棒性（+11 ~ +37 pp）
3	**保留路线层面的重估**	给局部修改设上限，卡住就交回研究员；不要加只会否决的审批角色
4	**沉淀并复用技能**	原语技能库让新任务成功率 21.3% → 40.0%，没有负迁移
5	**冻结为程序，必要时在线兜底**	执行快 5–11 倍；精密接触任务（网线插入）仍需在线智能体

一句话：把真机试错变成可复用、可冻结的机器人代码，这条路可行，但「91% / 84%」需结合统计口径来看。